

Ειδικά Θέματα Αλγορίθμων: Μηχανική Μάθηση

Πρώτο Πακέτο Ασκήσεων

Παναγή Αντρέας

Α.Λ.Μ.Α.

Τμήμα Μαθηματικών
Εθνικόν και Καποδιστριακόν Πανεπιστήμιον Αθηνών
Αθήνα, Ελλάδα *

Μάρτιος 2024



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν
Πανεπιστήμιο Αθηνών

Περιεχόμενα

Άσκηση 1	2
Εκφώνηση	2
Λύση	2
Άσκηση 2	4
Εκφώνηση	4
Λύση	4
Άσκηση 3	6
Εκφώνηση	6
Λύση: Με κωδικοποίηση στο 0,1	6
Λύση: Με απόσταση Gowers	7
Απλή Υλοποίηση kNN	10

Άσκηση 1

Εκφώνηση

Δημιουργήστε μια συνάρτηση `knn_gen` για πρόβλεψη μιας γενικής διακριτής εξαρτημένης μεταβλητής με m κατηγορίες, και p ανεξάρτητες μεταβλητές. Η συνάρτηση θα είναι της μορφής `knn_gen(xtrain, xtest, ytrain, k)`, όπου:

- `xtrain` ο πίνακας ή `data.frame` τιμών των ανεξάρτητων μεταβλητών στο training set
- `ytrain` το διάνυσμα κατηγοριών της εξαρτημένης μεταβλητής στο training set
- `xtest` ο πίνακας ή `data.frame` τιμών των ανεξάρτητων μεταβλητών στο test set
- k η τάξη του αλγορίθμου k NN (πληθάριθμος του συνόλου γειτονικών σημείων)

Η συνάρτηση επιστρέφει το διάνυσμα προβλεπόμενων τιμών της Y στο test set.

Υποθέστε ότι όλες οι ανεξάρτητες μεταβλητές είναι συνεχείς. Για τον υπολογισμό της απόστασης χρησιμοποιήστε την Ευκλείδεια απόσταση. Ένα πρόβλημα στην πολυδιάστατη περίπτωση είναι ότι ο υπολογισμός της απόστασης δύο σημείων στο χώρο των X δεν είναι ανεξάρτητος από τις μονάδες μέτρησης, με αποτέλεσμα οι μεταβλητές που έχουν γενικά μεγάλες τιμές να συνεισφέρουν πολύ περισσότερο στον υπολογισμό της απόστασης από μεταβλητές που έχουν γενικά μικρές τιμές, και επομένως η εύρεση των πλησιέστερων γειτόνων να εξαρτάται επίσης από τις μονάδες μέτρησης, που δεν είναι σωστό.

Ένας απλός τρόπος να αποφευχθεί αυτό το πρόβλημα είναι να γίνει κανονικοποίηση των ανεξάρτητων μεταβλητών πριν την εφαρμογή της μεθόδου. Αν έχουμε μια οικογένεια διανυσμάτων $x_1(1, \dots, m), \dots, x_p(1, \dots, m)$, μια απλή κανονικοποίηση είναι η $\frac{x_j(i) - \min_{1 \leq k \leq p} \{x_k(i)\}}{\max_{1 \leq k \leq p} \{x_k(i)\} - \min_{1 \leq k \leq p} \{x_k(i)\}}$ για κάθε συντεταγμένη $i \in [m]$, για κάθε διάνυσμα $j \in [p]$.

Επίσης μπορεί να γίνει η κλασική από τη στατιστική τυποποίηση, όπου το τυποποιημένο διάνυσμα είναι $\tilde{x} = (x - m)/s$ όπου m ο δειγματικός μέσος και s η δειγματική τυπική απόκλιση του x .

Λύση

Έστω ότι θέλουμε να υπολογίσουμε για κάθε διάνυσμα του training set (p_1 το πλήθος διανύσματα), τα k πλησιέστερα διανύσματα που βρίσκονται στο training set (το οποίο περιέχει p_2 το πλήθος διανύσματα).

Στην εν λόγω εργασία για το υπολογισμό της παραπάνω διεργασίας με χρήση `divide and conquer` (διαίρει και κυρίευε), παρουσιάζεται μια υλοποίηση σε χρόνο $O(p_1 p_2 \log k)$ αντί του κλασικού $O(p_1 p_2 \log p_2)$ (με απλή χρήση `sorting` των αποστάσεων).

Για κάθε διάνυσμα του test set κρατάμε τις τιμές των k πλησιέστερων διανυσμάτων σε μια δομή δεδομένων η οποία που καλείτε `σορός` (μεγίστου), αφού μπορούμε να εκμεταλευτούμε την μονοτονία των αριθμών αυτών.

```
[5]: import numpy as np
import heapq
from collections import Counter
```

```

def knn_gen(xtrain, xtest, ytrain, k):
    # Convert to NumPy arrays
    xtrain = np.asarray(xtrain, dtype=float)
    xtest = np.asarray(xtest, dtype=float)
    ytrain = np.asarray(ytrain)

    # Normalization
    mini = xtrain.min(axis=0)
    maxi = xtrain.max(axis=0)
    diff = np.where(maxi - mini == 0, 1.0, maxi - mini)

    xtrain_n = (xtrain - mini) / diff
    xtest_n = (xtest - mini) / diff

    predictions = []

    for i in range(xtest_n.shape[0]):
        max_heap = [] # Max-heap to keep track of k smallest distances

        for j in range(xtrain_n.shape[0]):
            dist = np.linalg.norm(xtrain_n[j] - xtest_n[i]) # Compute Euclidean
↪ distance

            if len(max_heap) < k:
                heapq.heappush(max_heap, (-dist, ytrain[j])) # Store negative
↪ distance

            elif -max_heap[0][0] > dist:
                heapq.heappushpop(max_heap, (-dist, ytrain[j])) # Replace
↪ largest if new dist is smaller

        # Extract k nearest labels
        kclass = [label for (_, label) in max_heap]

        # Majority vote
        predicted_class = Counter(kclass).most_common(1)[0][0]
        predictions.append(predicted_class)

    return predictions

```

Άσκηση 2

Εκφώνηση

Στην άσκηση αυτή καλείστε να εκτιμήσετε το μέσο σφάλμα πρόβλεψης του αλγορίθμου knn ως συνάρτηση της παραμέτρου k . Αυτό θα γίνει στο dataset South African Heart Disease. Η εξαρτημένη μεταβλητή είναι η chd που δείχνει την ύπαρξη ή όχι καρδιαγγειακής νόσου. Χρησιμοποιήστε μόνο τις ποσοτικές ανεξάρτητες μεταβλητές (sdp , $tobacco$, ldl , $adiposity$, $obesity$, $alcohol$, age).

1. Κατεβάστε το South African Heart dataset από την ιστοσελίδα του ESLR: <https://hastie.su.domains/ElemStatLearn/>. Το αρχείο είναι σε μορφή text (comma separated) και από αυτό θα δημιουργήσετε το κατάλληλο dataset σε python.
2. Χωρίστε το dataset των 463 παρατηρήσεων σε δύο υποσύνολα: ένα training set μεγέθους $N_1 = 400$ και ένα test set μεγέθους $N_2 = 63$, κάνοντας τη διαμέριση με τυχαίο τρόπο.
3. Θεωρήστε τιμές του $k = 1, 10, 20, 50, 100, 200, 300$
4. Για κάθε τιμή εφαρμόστε τη συνάρτηση `knn_gen` σε όλες τις παρατηρήσεις του test set, χρησιμοποιώντας ως δεδομένα τις N_1 παρατηρήσεις του training set. Έστω $y^1, \dots, y^{N_2}$ οι προβλέψεις για τα σημεία $x_1, \dots, x_{N_2}$ του test set και εκτιμήστε το μέσο σφάλμα (misclassification rate).

Δημιουργήστε ένα γράφημα του μέσου σφάλματος πρόβλεψης ως συνάρτησης του k και εξηγήστε τι παρατηρείτε σχετικά με το bias-variance tradeoff.

Λύση

```
[ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
%matplotlib inline

# Load the dataset
df_ma = pd.read_csv("data/sa_heart_dataset.csv")

# Data preprocessing
df_ma = df_ma.drop(columns=['famhist'])

train_df, test_df = train_test_split(df_ma, train_size=400, test_size=62,
    random_state=42)

train = train_df.to_numpy()
test = test_df.to_numpy()

xtrain = train[:, 1:-1]
xtest = test[:, 1:-1]
ytrain = train[:, -1]
ytest = test[:, -1]

# Define the range of k values to test
K=[1,11,21,51,101,201,301]
accuracies = [] # This list will store accuracy for each k
```

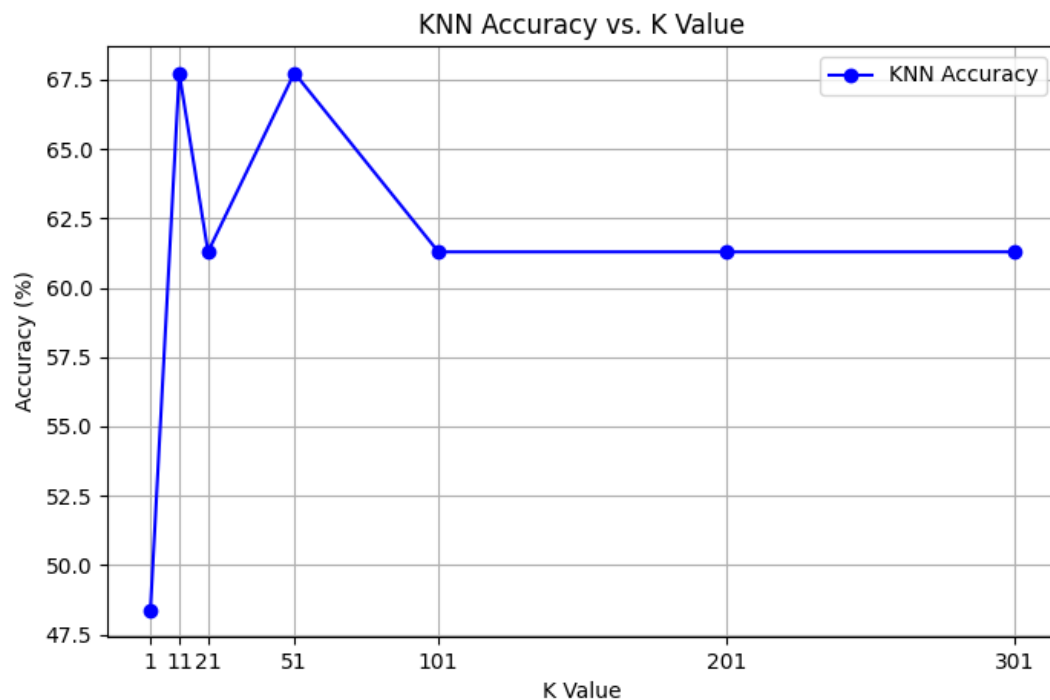
```

for k in K:
    predictions = knn_gen(xtrain, xtest, ytrain, k) # Your knn_gen function
    acc = np.mean(np.array(predictions) == ytest) * 100
    accuracies.append(acc)
    print(f"Accuracy for k={k}: {acc:.2f}%")

# Plot
plt.figure(figsize=(8, 5))
plt.plot(K, accuracies, marker='o', linestyle='-', color='b', label="KNN_
Accuracy")
plt.xlabel("K Value")
plt.ylabel("Accuracy (%)")
plt.title("KNN Accuracy vs. K Value")
plt.xticks(K) # Label the x-axis ticks using the values in K
plt.grid(True)
plt.legend()
plt.savefig("knn_accuracy_plot.png") # Save the plot as a PNG file
plt.show()

```

Accuracy for k=1: 48.39%
 Accuracy for k=11: 67.74%
 Accuracy for k=21: 61.29%
 Accuracy for k=51: 67.74%
 Accuracy for k=101: 61.29%
 Accuracy for k=201: 61.29%
 Accuracy for k=301: 61.29%



Άσκηση 3

Εκφώνηση

Για να εφαρμοστεί η μέθοδος knn σε περιπτώσεις που υπάρχουν κατηγορικές ανεξάρτητες μεταβλητές, είτε αυτές πρέπει πρώτα να κωδικοποιηθούν κατάλληλα είτε να χρησιμοποιηθεί μια εναλλακτική μετρική απόστασης. Μια συνηθισμένη μετρική είναι η απόσταση Gowers.

Για την απόσταση Gowers υπάρχουν συναρτήσεις σε python.

Τροποποιήστε τη συνάρτηση knn_gen στην knn_gen_gowers, η οποία χρησιμοποιεί την απόσταση Gowers αντί για την Ευκλείδεια απόσταση, και εφαρμόστε την στο South African Heart Disease dataset αυτή τη φορά με όλες τις ανεξάρτητες μεταβλητές.

Εκτιμήστε σε ποιο βαθμό βελτιώνεται το μέσο σφάλμα πρόβλεψης σχετικά με την Άσκηση 2.

Λύση: Με κωδικοποίηση στο 0,1

Παρατηρούμε ότι στο συγκεκριμένο data set, η στήλη famhist (feature) δεν περιέχει αριθμητικές τιμές αλλά strings τα οποία παίρνουν τιμές Present και Absent.

Στην παρακάτω λύση κωδικοποιούμε τις τιμές Present σε 1 και την Absent σε 0 ώστε να μπορούμε να χρησιμοποιήσουμε τον αλγόριθμο της άσκησης 1.

```
[6]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load the dataset
df_ma = pd.read_csv("data/sa_heart_dataset.csv")

# Data preprocessing
df_ma['famhist'] = df_ma['famhist'].map({'Present': 1, 'Absent': 0})

train_df, test_df = train_test_split(df_ma, train_size=400, test_size=62,
                                     random_state=42)

train = train_df.to_numpy()
test = test_df.to_numpy()

xtrain = train[:, 1:-1]
xtest = test[:, 1:-1]
ytrain = train[:, -1]
ytest = test[:, -1]

# Define the range of k values to test
K=[1,11,21,51,101,201,301]
accuracies = [] # This list will store accuracy for each k

for k in K:
    predictions = knn_gen(xtrain, xtest, ytrain, k) # Your knn_gen function
    acc = np.mean(np.array(predictions) == ytest) * 100
    accuracies.append(acc)
```

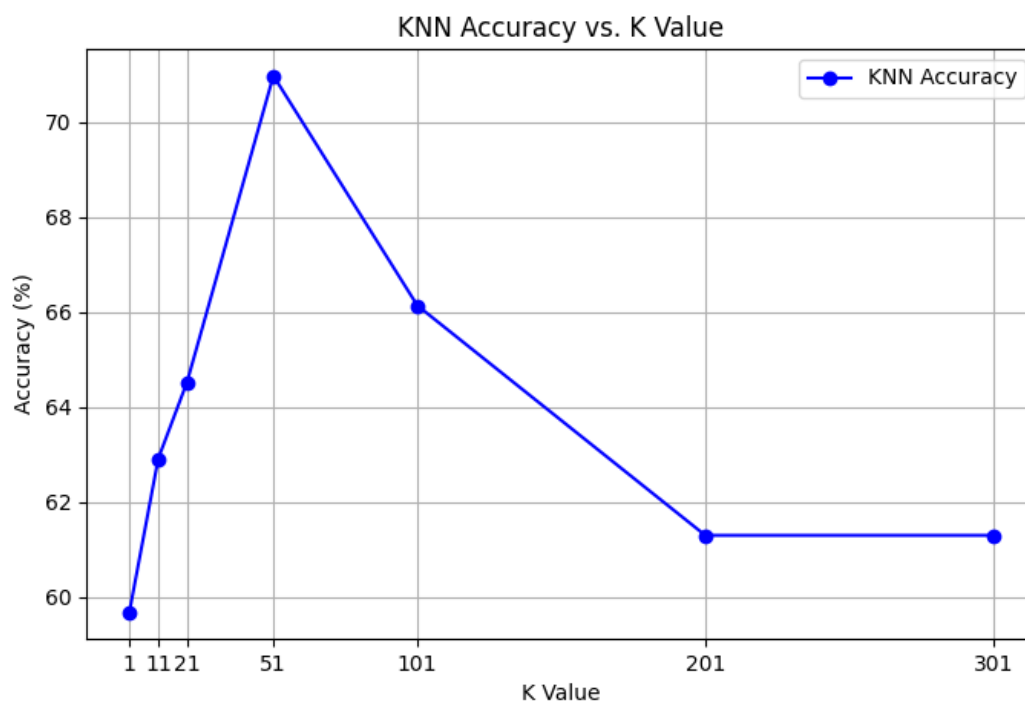
```

print(f"Accuracy for k={k}: {acc:.2f}%")

# Plot
plt.figure(figsize=(8, 5))
plt.plot(K, accuracies, marker='o', linestyle='-', color='b', label="KNN_Accuracy")
plt.xlabel("K Value")
plt.ylabel("Accuracy (%)")
plt.title("KNN Accuracy vs. K Value")
plt.xticks(K) # Label the x-axis ticks using the values in K
plt.grid(True)
plt.legend()
plt.show()

```

Accuracy for k=1: 59.68%
 Accuracy for k=11: 62.90%
 Accuracy for k=21: 64.52%
 Accuracy for k=51: 70.97%
 Accuracy for k=101: 66.13%
 Accuracy for k=201: 61.29%
 Accuracy for k=301: 61.29%



Λύση: Με απόσταση Gowers

```

[17]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

```



```

import gower # Import the gower package
import heapq
from collections import Counter

def knn_gen_gower(xtrain, xtest, ytrain, k):
    xtrain = np.asarray(xtrain)
    xtest = np.asarray(xtest)
    ytrain = np.asarray(ytrain)

    predictions = []

    # Compute Gower distances using the correct function
    distances = gower.gower_matrix(xtest, xtrain) # Shape: (n_test, n_train)

    for i in range(xtest.shape[0]):
        max_heap = [] # Max-heap to store k smallest distances

        for j in range(xtrain.shape[0]):
            dist = distances[i, j]

            if len(max_heap) < k:
                heapq.heappush(max_heap, (-dist, ytrain[j])) # Store negative
↪for max-heap behavior
            elif -max_heap[0][0] > dist:
                heapq.heappushpop(max_heap, (-dist, ytrain[j])) # Replace
↪largest if new dist is smaller

            # Extract k nearest labels
            kclass = [label for _, label in max_heap]

            # Majority voting
            predicted_class = Counter(kclass).most_common(1)[0][0]
            predictions.append(predicted_class)

    return predictions

# Load dataset
df_ma = pd.read_csv("data/sa_heart_dataset.csv")

# Data preprocessing
train_df, test_df = train_test_split(df_ma, train_size=400, test_size=62,
↪random_state=42)

xtrain = train_df.iloc[:, 1:-1].values
xtest = test_df.iloc[:, 1:-1].values
ytrain = train_df.iloc[:, -1].values
ytest = test_df.iloc[:, -1].values

# Define the range of k values to test
K = [1, 11, 21, 51, 101, 201, 301]
accuracies = [] # Store accuracy for each k

```

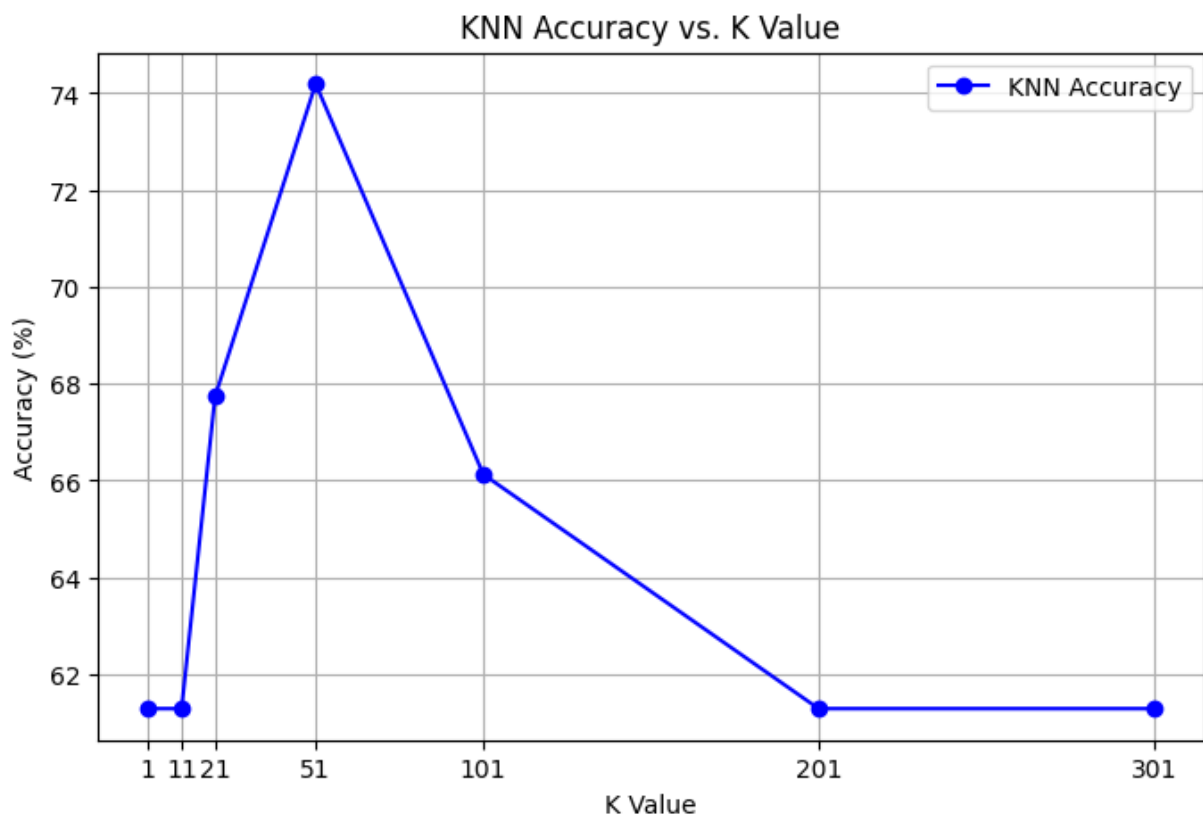
```

for k in K:
    predictions = knn_gen_gower(xtrain, xtest, ytrain, k) # Call knn function
    acc = np.mean(np.array(predictions) == ytest) * 100
    accuracies.append(acc)
    print(f"Accuracy for k={k}: {acc:.2f}%")

# Plot results
plt.figure(figsize=(8, 5))
plt.plot(K, accuracies, marker='o', linestyle='-', color='b', label="KNN_
Accuracy")
plt.xlabel("K Value")
plt.ylabel("Accuracy (%)")
plt.title("KNN Accuracy vs. K Value")
plt.xticks(K) # Label x-axis with K values
plt.grid(True)
plt.legend()
plt.savefig("knn_accuracy_plot.png") # Save the plot
plt.show()

```

Accuracy for k=1: 61.29%
 Accuracy for k=11: 61.29%
 Accuracy for k=21: 67.74%
 Accuracy for k=51: 74.19%
 Accuracy for k=101: 66.13%
 Accuracy for k=201: 61.29%
 Accuracy for k=301: 61.29%



Απλή Υλοποίηση k NN

Για να υλοποιήσουμε πολλές ανάγκες είναι προτιμότερο να χρησιμοποιούμε έτοιμες εντολές και πακέτα (παρά να τα δημιουργούμε) αφού είναι γραμμένα με βέλτιστο τρόπο, ώστε να εκτελούνται πολύ πιο γρήγορα.

Κλασσικό παράδειγμα αποτελεί και η υλοποίηση της k NN καθώς στην πράξη η απλή εκτέλεση ενός (θεωρητικά πιο χρονοβόρου) sort ή καλύτερα argument sort δίνει γρηγορότερο χρόνο εκτέλεσης από την θεωρητικά καλύτερη προσέγγιση με το divide and conquer.

Παρακάτω παρατίθεται ο κώδικας για την απλή (θεωρητικά πιο αργή) αλλά πρακτικά γρηγορότερη εκτέλεση του k NN.

```
[8]: import numpy as np
from collections import Counter

def knn_gen_s(xtrain, xtest, ytrain, k):
    # Convert to NumPy arrays
    xtrain = np.asarray(xtrain, dtype=float)
    xtest = np.asarray(xtest, dtype=float)
    ytrain = np.asarray(ytrain)

    # Normalization
    mini = xtrain.min(axis=0)
    maxi = xtrain.max(axis=0)
    diff = np.where(maxi - mini == 0, 1.0, maxi - mini)

    xtrain_n = (xtrain - mini) / diff
    xtest_n = (xtest - mini) / diff

    predictions = []

    for i in range(xtest_n.shape[0]):
        # Compute Euclidean distances
        distances = np.linalg.norm(xtrain_n - xtest_n[i], axis=1)

        # Find the indices of the k smallest distances
        kmin_indices = np.argsort(distances)[:k]

        # Get corresponding labels
        kclass = ytrain[kmin_indices]

        # Majority voting
        predicted_class = Counter(kclass).most_common(1)[0][0]
        predictions.append(predicted_class)

    return predictions
```